

## BITS PILANI WILP

# Software Engineering for Machine Learning (AIMLCZG546)

## Assignment II — Implementation, Code Quality & Quality Assurance

**Project:** Loan Approval Risk Service — automated consumer credit underwriting  
**Group Number:** 84 · **Date:** August 2026 · **Builds on:** Assignment I (Group 84)

### Group Members & Contributions

| Sl. | BITS ID     | Name                 | Qualitative Contribution                                                                                                     | %   |
|-----|-------------|----------------------|------------------------------------------------------------------------------------------------------------------------------|-----|
| 1   | 2025AA05710 | Singh Pritesh        | Refactoring to the <code>loan_risk</code> package, error handling & structured logging, lint/format toolchain, quality gates | 100 |
| 2   | 2025AA05368 | Gangera Tushar       | Research-vs-production analysis, data-quality metrics, schema contract and data-validation test suite                        | 100 |
| 3   | 2025AB05154 | Gangam Shuba Nandini | ML behavioural tests (training & inference), model-quality metrics, calibration and drift monitoring                         | 100 |
| 4   | 2025AA05574 | Shaifali Garg        | FastAPI design & implementation, integration tests, production experimentation and security analysis                         | 100 |

### Executive Summary

Assignment I delivered a working prototype: a four-filter pipe-and-filter pipeline behind a single FastAPI endpoint. This assignment turns that prototype into a system that could be handed to an operations team. The scoring logic was rebuilt as an installable package (`src/loan_risk/`) with one cohesive class per responsibility; a research notebook is retained unchanged beside its engineered counterpart so the difference is evidence rather than assertion; error handling and structured JSON logging were added across the ingestion, validation and training paths; the whole codebase was put under `isort`, `black`, `flake8` and `pylint`; and the API was redesigned with versioned resources, a bounded batch endpoint and differentiated status codes.

On the QA side, 33 pytest tests were written across four distinct test types, including the ML-specific families the assignment calls for — overfit-a-small-batch, loss-decreases, output shape/range, directional and invariance. Four model-quality metrics and four data-quality metrics are measured and gated. The measured headline numbers are: accuracy **0.9480**, F1 **0.9343**, ROC-AUC **0.9881**, Brier **0.0458**, mean latency **10.29 ms** against a 150 ms SLA, flake8 **45** → **0** violations and pylint **10.00/10**.

The test suite earned its keep before submission. The invariance test `test_prediction_is_invariant_to_repeated_calls` failed on the first run: with `n_jobs=-1` the forest accumulated tree votes in non-deterministic thread order, so two identical requests could differ by  $\sim 5e-17$  — enough to flip an applicant sitting exactly on the 0.50 cut-off between APPROVED and DENIED. Section 7.3 documents the defect and the fix.

# Objective 1 — Implementation and Code Sharing

## 1. Refactoring with OOP and Functional Principles

The Assignment I code was a flat app/ folder in which `pipeline.py` held four module-level functions that each reached into a global settings object, and `main.py` called `joblib.load` directly. That is serviceable for a demo and awkward to test, extend or reason about. The refactor splits the system along responsibility boundaries, one module per concern:

| Module                               | Type                                          | Responsibility                                               | Paradigm                      |
|--------------------------------------|-----------------------------------------------|--------------------------------------------------------------|-------------------------------|
| <code>config.py</code>               | Settings et al.                               | Frozen dataclasses loaded from YAML; every threshold is data | OOP (immutable value objects) |
| <code>data/ingestion.py</code>       | DataIngestor                                  | The only component that reads training data from disk        | OOP                           |
| <code>data/validation.py</code>      | DataValidator                                 | Declarative schema contract; emits DataQualityReport         | OOP + declarative spec        |
| <code>features/engineering.py</code> | FeatureEngineer                               | Derived ratios; sklearn transformer wrapping pure functions  | Functional core, OOP shell    |
| <code>models/trainer.py</code>       | ModelTrainer                                  | train → evaluate → gate → persist lifecycle                  | OOP                           |
| <code>models/predictor.py</code>     | ModelRegistry, RiskPredictor                  | Artifact custody; the four scoring filters                   | OOP + pure filters            |
| <code>monitoring/drift.py</code>     | DriftMonitor                                  | PSI and KS drift against the frozen reference profile        | Functional core, OOP shell    |
| <code>api/</code>                    | <code>schemas.py</code> , <code>app.py</code> | HTTP contract, routing and exception→status mapping          | Declarative (Pydantic)        |

Figure 1: Module responsibility map of the `loan_risk` package.

### Where each paradigm was chosen, and why

- **Object-oriented** where there is genuine state or a lifecycle to own: `ModelRegistry` owns the loaded artifact, `ModelTrainer` owns the fitted pipeline and its metrics. Constructor injection of `Settings` replaced the global lookups, so any test can substitute a different configuration without patching a module.
- **Functional** where the operation is a transformation: `safe_ratio`, `compute_loan_to_income`, `assign_risk_tier` and `validate_business_rules` are pure — same input, same output, no mutation. `test_validation_filter_does_not_mutate_the_payload` and `test_transformer_does_not_mutate_its_input` assert that purity rather than trusting it.
- **Immutability** for configuration: the `Settings` aggregate is a frozen dataclass, so a request handler cannot quietly rewrite the decision threshold for every other request in the process.
- **Composition over duplication**: `FeatureEngineer` is a scikit-learn `TransformerMixin` and is therefore the *first step of the serialised pipeline*. Training and serving cannot disagree about feature definitions, because they execute the same pickled object.

The last point is the single most valuable structural change. In Assignment I the derived ratios were computed in `data/prepare_data.py` for training and again in `extract_features()` for serving — two copies of one definition, which is the textbook setup for training/serving skew. There is now one copy, and it travels inside the artifact.

```
if algorithm == "random_forest":
    estimator = RandomForestClassifier(
        n_estimators=model_cfg.n_estimators,
        max_depth=model_cfg.max_depth,
        min_samples_leaf=model_cfg.min_samples_leaf,
        random_state=model_cfg.random_state,
        # n_jobs=1 is a *correctness* choice, not a performance
        # oversight. With n_jobs=-1 the per-tree vote accumulation
        # happens in non-deterministic thread order, so two identical
        # requests can differ at ~1e-16 -- enough to flip an applicant
        # sitting exactly on the 0.50 cut-off. Regulated lending
        # decisions must be bit-reproducible for audit, and inference
        # is already ~2 ms per request, so there is nothing to buy.
```

```

        # Caught by tests/test_model_inference.py::
        # test_prediction_is_invariant_to_repeated_calls.
        n_jobs=1,
    )
    steps = [("features", FeatureEngineer()), ("model", estimator)]
    ... "pipeline_built", extra={"algorithm": algorithm, "n_steps": len(steps)}
    )
    return Pipeline(steps)

```

Listing 1: the estimator is assembled as a Pipeline whose first stage is the serving-time feature transformer.

## 2. Research Code vs Production Code

The component chosen for this comparison is **feature engineering**. Both artefacts are in the repository and both compute the same two ratios:

- **Research:** notebooks/research\_prototype.ipynb (also exported verbatim as legacy/research\_feature\_prototype.py so the linters can be pointed at it).
- **Production:** src/loan\_risk/features/engineering.py.

The research notebook is not a strawman — it is what genuinely useful exploratory code looks like. It answered the question it was written for. The point of the table below is that every one of its shortcuts is a defect only once the code has to run unattended, and each has a specific engineered answer.

| # | Research code (notebook)                               | Consequence in production                            | Production answer                                                                              |
|---|--------------------------------------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------|
| 1 | Hard-coded absolute path C:\Users\analyst\Desktop /... | Runs on exactly one machine                          | Settings.path() resolves every path from config.yaml                                           |
| 2 | train_test_split with no random_state                  | Metrics move every run; results are unauditale       | random_state from config; test_training_is_reproducible asserts bit-identical output           |
| 3 | LoanAmount/AnnualIncome with no guard                  | inf / ZeroDivisionError on a zero-income application | safe_ratio() applies +1 smoothing; a finiteness check raises FeatureEngineeringError           |
| 4 | Feature list retyped by hand in the notebook           | Training/serving skew as soon as one copy changes    | One FEATURE_ORDER contract, sourced from config and re-imposed on every transform              |
| 5 | Dead variables (tmp, ratio2, x)                        | Reader cannot tell what is load-bearing              | Every symbol is used; flake8 F401/F841 enforce it                                              |
| 6 | except: pass                                           | Failures are silent                                  | Typed exception hierarchy; every handler logs at a level and re-raises                         |
| 7 | No logging, no types, no docstrings                    | Nothing is observable; behaviour is undocumented     | Structured JSON logging, full type hints, docstring on every public callable (pylint 10.00/10) |
| 8 | Logic lives in notebook cells                          | Cannot be imported, reused or unit-tested            | Importable package; 14 unit tests cover this module alone                                      |
| 9 | print() for diagnostics                                | Unstructured output, unusable by a log aggregator    | logger.info(..., extra={...}) emitting one JSON object per event                               |

Figure 2: Defect-by-defect mapping from the research notebook to its production counterpart.

### Side-by-side: the same computation, twice

**Research** — notebooks/research\_prototype.ipynb

```

df['LoanToIncomeRatio'] = df['LoanAmount']/df['AnnualIncome'] # NOTE: blows up when income is 0
df['SavingsToLoanRatio'] = df['SavingsAccountBalance']/df['LoanAmount']
df['ratio2'] = df['MonthlyDebtPayments']*12/df['AnnualIncome']
df['x'] = df['CreditScore']/850
FEATURES=['Age', 'AnnualIncome', 'CreditScore', 'LoanAmount', 'LoanToIncomeRatio', ...]

def get_ratio(a,b) :
    return a/b # no zero guard, no types, no docstring

```

**Production** — src/loan\_risk/features/engineering.py

```

def safe_ratio(
    numerator: pd.Series | float, denominator: pd.Series | float
) -> pd.Series | float:
    """Divide with +1 smoothing so the denominator can never be zero.

    Pure function -- no I/O, no globals -- which is exactly why it can be
    property-tested in isolation (see ``tests/test_unit_features.py``).
    """
    return numerator / (denominator + _EPSILON)

def compute_loan_to_income(loan_amount, annual_income):
    """Leverage ratio: how many times the annual income is being borrowed."""
    return safe_ratio(loan_amount, annual_income)

def compute_savings_to_loan(savings_balance, loan_amount):
    """Cushion ratio: liquid savings held against the requested principal."""
    return safe_ratio(savings_balance, loan_amount)

def transform(self, X: pd.DataFrame) -> pd.DataFrame: # noqa: N803
    """Return a frame containing exactly ``feature_order``, in order.

    Raises:
        FeatureEngineeringError: if a required base column is absent or the
        derived values are not finite.
    """
    if not isinstance(X, pd.DataFrame):
        raise FeatureEngineeringError(
            f"Expected a pandas DataFrame, received {type(X).__name__}"
        )

    frame = X.copy()
    missing = [c for c in BASE_COLUMNS if c not in frame.columns]
    if missing:
        logger.error("feature_base_columns_missing", extra={"missing": missing})
        raise FeatureEngineeringError(f"Missing base columns: {missing}")

    frame["LoanToIncomeRatio"] = compute_loan_to_income(
        frame["LoanAmount"], frame["AnnualIncome"]
    )
    frame["SavingsToLoanRatio"] = compute_savings_to_loan(
        frame["SavingsAccountBalance"], frame["LoanAmount"]
    )

    derived = frame[["LoanToIncomeRatio", "SavingsToLoanRatio"]]
    if not np.isfinite(derived.to_numpy(dtype=float)).all():
        logger.error("derived_features_not_finite")
        raise FeatureEngineeringError(
            "Derived ratios contain NaN/inf -- check inputs for nulls."
        )

    out_of_range = int((frame["LoanToIncomeRatio"] > 5.0).sum())
    if out_of_range:
        # Recoverable: the model still scores these, but the operator should
        # know that unusually leveraged applications are arriving.
        logger.warning(
            "high_leverage_applications_detected",
            extra={"count": out_of_range, "threshold_loan_to_income": 5.0},
        )

    logger.info(
        "feature_engineering_completed",
        extra={"rows": int(len(frame)), "n_features": len(self.feature_order)},
    )
    return frame[self.feature_order]

```

Listing 2: the production transformer — declared contract, defensive division, explicit failure modes, structured logging, and a sklearn interface so it can be serialised with the model.

### 3. Error Handling and Logging

A typed exception hierarchy rooted at `LoanRiskError` lets the API layer tell our failures apart from unexpected ones and map each to the right status code, instead of returning a blanket 500. Logging is structured JSON on stdout — the same records that help a developer locally become the monitoring substrate in production without a second instrumentation path.

| Level | Policy in this codebase                | Example event                                                                                                    |
|-------|----------------------------------------|------------------------------------------------------------------------------------------------------------------|
| INFO  | Normal lifecycle events worth auditing | <code>data_ingestion_completed</code> , <code>model_artifact_saved</code> , <code>loan_application_scored</code> |

| Level   | Policy in this codebase                                                                                       | Example event                                                                                                |
|---------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| WARNING | Recoverable anomaly: a fallback was taken, a soft data rule was breached, or a request was rejected by policy | duplicate_rows_detected,<br>data_quality_violations_detected,<br>business_rule_rejected, data_drift_detected |
| ERROR   | The operation failed and the caller receives an error                                                         | data_source_missing,<br>feature_contract_violation,<br>quality_gate_breached, model_training_failed          |

Figure 3: Log-level policy, applied consistently across all modules.

### The three critical functions

**Critical function 1** — `DataIngestor.load()`: every I/O failure mode (missing file, empty file, unparseable CSV, zero rows) is caught, logged and re-raised as a single `DataIngestionError`, so callers never have to catch `OSError`. Duplicate rows are a **WARNING** — the run continues, but the operator is told.

```
def load(self, path: Path | str | None = None) -> pd.DataFrame:
    """Read a CSV into a dataframe.

    Raises:
        DataIngestionError: file missing, empty, or unparseable.
    """
    source = Path(path) if path else self.config.path("processed_data")
    logger.info("data_ingestion_started", extra={"source": str(source)})

    if not source.exists():
        logger.error(
            "data_source_missing",
            extra={"source": str(source), "hint": "run scripts/build_dataset.py"},
        )
        raise DataIngestionError(f"Dataset not found at {source}")

    try:
        frame = pd.read_csv(source)
    except pd.errors.EmptyDataError as exc:
        logger.error("data_source_empty", extra={"source": str(source)})
        raise DataIngestionError(f"Dataset at {source} is empty") from exc
    except (pd.errors.ParserError, UnicodeDecodeError, OSError) as exc:
        logger.error(
            "data_source_unreadable",
            extra={"source": str(source), "error": str(exc)},
        )
        raise DataIngestionError(f"Could not parse {source}: {exc}") from exc

    if frame.empty:
        logger.error("data_source_zero_rows", extra={"source": str(source)})
        raise DataIngestionError(f"Dataset at {source} contains zero rows")

    duplicates = int(frame.duplicated().sum())
    if duplicates:
        # Recoverable: we keep going but the operator must know.
        logger.warning(
            "duplicate_rows_detected",
            extra={"duplicate_rows": duplicates, "total_rows": len(frame)},
        )

    logger.info(
        "data_ingestion_completed",
        extra={"rows": int(frame.shape[0]), "columns": int(frame.shape[1])},
    )
    return frame
```

**Critical function 2** — `DataValidator.validate()`: soft breaches accumulate and log at **WARNING**; in `strict=True` mode (used by the training entrypoint) a breach logs at **ERROR** and raises `SchemaValidationError` carrying the full violation list. Training on silently corrupt data is the failure this prevents.

```
def validate(self, frame: pd.DataFrame, strict: bool = False) -> DataQualityReport:
    if frame is None or frame.empty:
        logger.error("validation_called_on_empty_frame")
        raise SchemaValidationError("Cannot validate an empty dataframe.")

    violations: List[str] = []
    conforming_columns = 0
    for spec in self.schema:
        findings = self._check_column(frame, spec)
        violations.extend(findings)
        conforming_columns += int(not findings)
```

```

# ---- DQ metric 1: schema conformance rate -----
conformance = conforming_columns / len(self.schema)

# ---- DQ metric 2: missing-value fractions -----
null_fractions = frame.isna().mean()
overall_missing = float(frame.isna().to_numpy().mean())
worst_column = str(null_fractions.idxmax()) if len(null_fractions) else None
worst_fraction = float(null_fractions.max()) if len(null_fractions) else 0.0

if overall_missing > self.max_missing_fraction:
    violations.append(
        f"missing_fraction_exceeded:{overall_missing:.4f}"
        f">{self.max_missing_fraction}"
    )

report = DataQualityReport(
    n_rows=int(frame.shape[0]),
    n_columns=int(frame.shape[1]),
    schema_conformance_rate=conformance,
    missing_value_fraction=overall_missing,
    worst_column_missing_fraction=worst_fraction,
    worst_column=worst_column,
    violations=violations,
)

if violations:
    logger.warning(
        "data_quality_violations_detected",
        extra={
            "n_violations": len(violations),
            "violations": violations[:10],
            "schema_conformance_rate": round(conformance, 4),
        },
    )
    if strict:
        logger.error(
            "data_quality_gate_failed",
            extra={"n_violations": len(violations)},
        )
        raise SchemaValidationError(
            f"{len(violations)} schema violation(s) detected.", violations
        )
else:
    logger.info(
        "data_quality_gate_passed",
        extra={
            "rows": report.n_rows,
            "schema_conformance_rate": 1.0,
            "missing_value_fraction": round(overall_missing, 6),
        },
    )
return report

```

**Critical function 3** — `ModelTrainer.train()` / `enforce_quality_gates()`: length mismatch and a single-class target are rejected before fitting; a fit exception is wrapped as `ModelTrainingError`; and a model that misses a release gate is logged at `ERROR` and refuses to ship.

```

def enforce_quality_gates(self, metrics: Optional[ModelMetrics] = None) -> None:
    metrics = metrics or self.metrics
    if metrics is None:
        raise ModelTrainingError("No metrics available to gate on.")

    gates = self.config.gates
    breaches = []
    if metrics.accuracy < gates.min_accuracy:
        breaches.append(f"accuracy {metrics.accuracy:.4f} < {gates.min_accuracy}")
    if metrics.f1 < gates.min_f1:
        breaches.append(f"f1 {metrics.f1:.4f} < {gates.min_f1}")
    if metrics.roc_auc < gates.min_roc_auc:
        breaches.append(f"roc_auc {metrics.roc_auc:.4f} < {gates.min_roc_auc}")
    if metrics.brier_score > gates.max_brier_score:
        breaches.append(f"brier {metrics.brier_score:.4f} > {gates.max_brier_score}")

    if breaches:
        logger.error("quality_gate_breached", extra={"breaches": breaches})
        raise ModelTrainingError("Quality gates breached: " + "; ".join(breaches))
    logger.info("quality_gates_passed", extra=metrics.as_dict())

```

Sample of the real emitted log stream (reports/metrics/training\_log.txt):

```

{"timestamp": "2026-08-03T22:33:34", "level": "INFO", "logger": "loan_risk.data.ingestion", "message": "data_ingestion_c"}
{"timestamp": "2026-08-03T22:33:34", "level": "INFO", "logger": "loan_risk.data.validation", "message": "data_quality_ga"}
{"timestamp": "2026-08-03T22:33:40", "level": "INFO", "logger": "loan_risk.models.trainer", "message": "model_evaluated"}
{"timestamp": "2026-08-03T22:33:41", "level": "WARNING", "logger": "loan_risk.monitoring.drift", "message": "data_drift"}
{"timestamp": "2026-08-03T22:33:41", "level": "INFO", "logger": "loan_risk.models.predictor", "message": "loan_applicati"}

```

Listing 3: every record is one JSON object, so `latency_ms` or `risk_tier` can be aggregated directly by ELK/Splunk with no log parsing.

## 4. Code Formatting and Linting

Four tools are configured, each with a distinct job: `isort` orders imports, `black` owns formatting outright, `flake8` catches correctness and complexity issues formatting cannot, and `pylint` adds design-level checks. Configuration lives in `pyproject.toml` (`black`, `isort`, `pytest`, `pylint`) and `setup.cfg` (`flake8`, which still has no `pyproject` support). The line length is set to 90 in all four so they cannot fight each other.

### Method

The **before** snapshot was taken on the codebase as first written, together with the untouched research code in `legacy/`. It reported **45 flake8 violations** across the tree and **13 files** that `black` would reformat. The formatters were then run, and the two findings the formatters cannot fix were repaired by hand — one of which was a genuine design issue (C901: `DataValidator.validate` had a cyclomatic complexity of 11 against a budget of 10, and was split into `_check_column` and `_check_range`). Both reports are stored verbatim in `reports/lint/`.

| Check                          | Command                                                     | Before    | After      |
|--------------------------------|-------------------------------------------------------------|-----------|------------|
| flake8 violations              | <code>python -m flake8 src scripts tests legacy</code>      | 45        | 0          |
| black — files needing reformat | <code>python -m black --check src scripts tests</code>      | 13        | 0          |
| isort — import order           | <code>python -m isort --check-only src scripts tests</code> | clean     | clean      |
| pylint score                   | <code>python -m pylint src/loan_risk</code>                 | 9.72 / 10 | 10.00 / 10 |

Figure 4: Lint and format results, before and after. Raw reports: `reports/lint/01_before_flake8.txt` → `07_after_flake8.txt`.

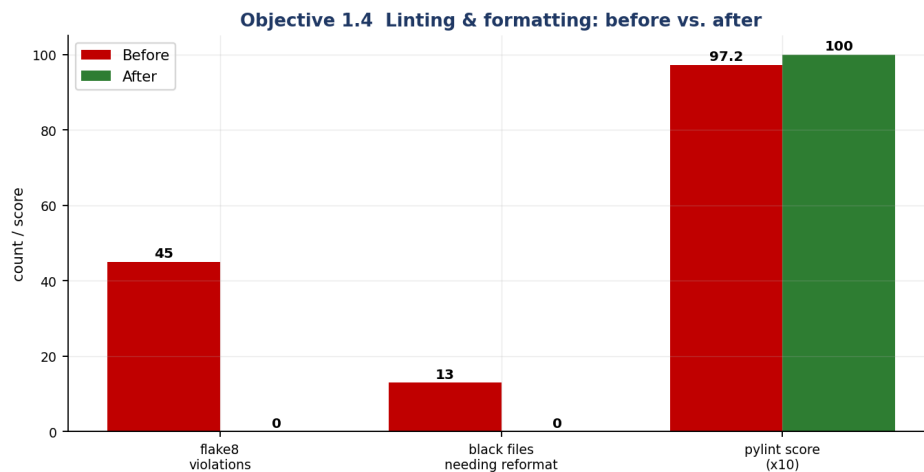


Figure 5: The same data as a chart. The pylint score is scaled  $\times 10$  to share the axis.

### Representative violations from the BEFORE report

```

legacy\research_feature_prototype.py:5:1: F401 'numpy as np' imported but unused
legacy\research_feature_prototype.py:5:20: E401 multiple imports on one line
legacy\research_feature_prototype.py:6:1: F401 'os' imported but unused
legacy\research_feature_prototype.py:6:1: F401 'sys' imported but unused
legacy\research_feature_prototype.py:6:1: F401 'json' imported but unused
legacy\research_feature_prototype.py:6:10: E401 multiple imports on one line
legacy\research_feature_prototype.py:12:91: E501 line too long (93 > 90 characters)
legacy\research_feature_prototype.py:14:7: E201 whitespace after '('
legacy\research_feature_prototype.py:14:17: E202 whitespace before ')'
legacy\research_feature_prototype.py:18:27: E231 missing whitespace after ','
legacy\research_feature_prototype.py:22:91: E501 line too long (101 > 90 characters)
legacy\research_feature_prototype.py:26:9: E225 missing whitespace around operator
legacy\research_feature_prototype.py:26:16: E231 missing whitespace after ','
legacy\research_feature_prototype.py:26:31: E231 missing whitespace after ','

```



Listing 4: extract from reports/lint/01\_before\_flake8.txt. E401 multiple imports per line, F401 unused import, E722 bare except, E231 missing whitespace, C901 too complex.

### AFTER — the same command on the production tree

```
$ python -m flake8 src scripts tests
$ python -m black --check src scripts tests
All done! 25 files would be left unchanged.
$ python -m isort --check-only src scripts tests
$ python -m pylint src/loan_risk
```

Your code has been rated at 10.00/10 (previous run: 9.72/10, +0.28)

Listing 5: flake8 and isort print nothing on success. legacy/ is deliberately excluded from black and isort — it is the evidence, and reformatting it would destroy the comparison.

## 5. REST API Design and Implementation

The service is a FastAPI application. Assignment I exposed a single unversioned /predict; the redesign applies the API-design practices the assignment asks for.

- **Versioned, resource-oriented paths.** /v1/predict, /v1/predict/batch, /v1/model/metadata. A breaking change can ship as /v2 without stranding callers. /health stays unversioned because orchestrator probes should never be tied to an API generation.
- **Explicit request/response schemas.** Every field is bounded with ge/le, and extra="forbid" rejects unknown keys — so a client typo like credit\_scr is a loud 422 rather than a silently defaulted feature.
- **Correct, differentiated status codes.** 200 scored · 422 schema violation · 400 business-rule rejection · 503 model unavailable · 404 unknown route. A policy rejection is a client error and must not be reported as a server fault.
- **A declared response\_model on every route,** so the OpenAPI contract is generated from the code and cannot drift from it.
- **A bounded batch endpoint** (1–100 applications). The cap is a denial-of-service control as much as an ergonomic one.
- **Exception handlers, not try/except in routes.** Domain exceptions are translated centrally, so internal messages and stack traces never reach the client — asserted by test\_error\_responses\_never\_leak\_internals.
- **Graceful degradation.** If the artifact is missing the process still starts, /health reports degraded, and scoring returns 503 — so an orchestrator keeps the previous replica serving instead of crash-looping.

REST API contract generated from the code (GET /openapi.json)

| Method | Path               | Summary        | Tag       | Documented status codes |
|--------|--------------------|----------------|-----------|-------------------------|
| GET    | /health            | Health         | ops       | 200                     |
| GET    | /v1/model/metadata | Model Metadata | ops       | 200                     |
| POST   | /v1/predict        | Predict        | inference | 200, 400, 422, 503      |
| POST   | /v1/predict/batch  | Predict Batch  | inference | 200, 400, 422, 503      |

Figure 6: The endpoint table, generated from the live service's own /openapi.json.

```
@app.get("/health", response_model=HealthResponse, tags=["ops"])
def health() -> HealthResponse:
    """Liveness/readiness probe for Kubernetes."""
    return HealthResponse(
        status="healthy" if registry.is_loaded else "degraded",
        model_loaded=registry.is_loaded,
        model_version=registry.version,
        environment=settings.env,
        api_version=settings.version,
    )

def predict(application: LoanApplication) -> RiskAssessmentResponse:
    """Score a single loan application."""
    assessment = predictor.predict(application.to_model_payload())
```



```
return RiskAssessmentResponse(**assessment.as_dict())
```

Listing 6: route definitions — documented status codes, declared response models, tags for the Swagger grouping.

### **The application: generated Swagger UI**

FastAPI derives the interactive documentation from the same type annotations that enforce validation at runtime, so the page below is not a hand-maintained artefact that can go stale — it *is* the contract. The screenshots are of the running service at `http://127.0.0.1:8077/docs`.

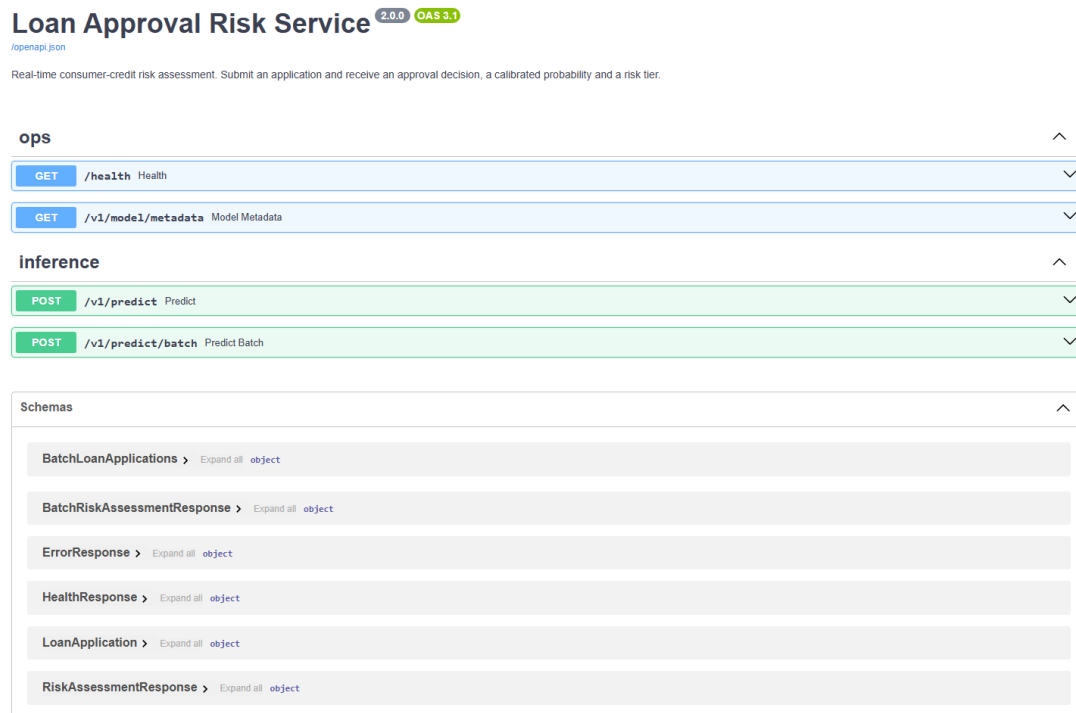


Figure 7: Swagger UI at /docs. Routes are grouped by tag (*ops*, *inference*) and all six request/response models are published under Schemas.

## Loan Approval Risk Service 2.0.0 OAS 3.1

[openapi.json](#)

Real-time consumer-credit risk assessment. Submit an application and receive an approval decision, a calibrated probability and a risk tier.

### ops

|     |                    |                |
|-----|--------------------|----------------|
| GET | /health            | Health         |
| GET | /v1/model/metadata | Model Metadata |

### inference

**POST** /v1/predict Predict

Score a single loan application.

**Parameters** Try it out

No parameters

**Request body** required application/json

Example Value | Schema

```
{
  "age": 45,
  "annual_income": 95000,
  "bankruptcy_history": 0,
  "checking_account_balance": 8000,
  "credit_card_utilization_rate": 0.25,
  "credit_score": 720,
  "debt_to_income_ratio": 0.15,
  "education_level": "graduate",
  "employment_status": 0,
  "job_tenure": 10,
  "length_of_credit_history": 15,
  "loan_amount": 75000,
  "loan_duration": 60,
  "loan_purpose": "other",
  "monthly_debt_payments": 400,
  "net_worth": 250000,
  "payment_history": 20,
  "previous_loan_defaults": 0,
  "savings_account_balance": 35000,
  "total_liabilities": 30000
}
```

**Responses**

| Code | Description                                                                                                                                                               | Links    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 200  | Successful Response                                                                                                                                                       | No links |
|      | Media type: <span>application/json</span><br>Controls: Accept header.                                                                                                     |          |
|      | Example Value   Schema                                                                                                                                                    |          |
|      | <pre>{   "is_approved": true,   "probability": 0.8,   "risk_tier": "low",   "net_worth": 0,   "debt_to_income": 0,   "latency_ms": 0,   "model_version": "string" }</pre> |          |
| 400  | Business-rule rejection                                                                                                                                                   | No links |
|      | Media type: <span>application/json</span>                                                                                                                                 |          |
|      | Example Value   Schema                                                                                                                                                    |          |
|      | <pre>{   "detail": "string",   "error_type": "string" }</pre>                                                                                                             |          |
| 422  | Schema validation failure                                                                                                                                                 | No links |
|      | Media type: <span>application/json</span>                                                                                                                                 |          |
|      | Example Value   Schema                                                                                                                                                    |          |
|      | <pre>{   "detail": "string",   "error_type": "string" }</pre>                                                                                                             |          |
| 503  | Model unavailable                                                                                                                                                         | No links |
|      | Media type: <span>application/json</span>                                                                                                                                 |          |
|      | Example Value   Schema                                                                                                                                                    |          |
|      | <pre>{   "detail": "string",   "error_type": "string" }</pre>                                                                                                             |          |

**POST** /v1/predict/batch Predict Batch

Figure 8: POST /v1/predict expanded. The worked example payload comes from the schema itself, and all four documented outcomes are published to the client: 200 scored, 400 business-rule rejection, 422 schema validation failure, 503 model unavailable.

### Verification against a running service

The figures below are the recorded transcript of real HTTP calls issued against the same live uvicorn process (reports/metrics/api\_transcript.json).

**Live service: health probe and two scored applications (HTTP 200)****GET /health****HTTP 200***Health probe*

```
request
(no request body)

response
{
  "status": "healthy",
  "model_loaded": true,
  "model_version": "2.0.0",
  "environment": "production",
  "api_version": "2.0.0"
}
```

**POST /v1/predict****HTTP 200***Low-risk applicant (200)*

```
request
{"age": 45, "annual_income": 95000, "credit_score": 720, "employment_status": 0, "education_level": 4, "loan_amount": 25000, "loan_duration": 36, "monthly_debt_payments": 400, "credit_card_utilization_rate": 0.25, "debt_to_income_ratio": 0.15, "bankruptcy_history": 0, "loan_purpose": 3, "previous_loan_defaults": 0,

response
{
  "is_approved": true,
  "probability": 0.939,
  "risk_tier": "LOW",
  "net_worth": 220000,
  "debt_to_income": 0.15,
  "latency_ms": 8.29,
  "model_version": "2.0.0"
}
```

**POST /v1/predict****HTTP 200***High-risk applicant (200)*

```
request
{"age": 45, "annual_income": 95000, "credit_score": 540, "employment_status": 0, "education_level": 4, "loan_amount": 90000, "loan_duration": 36, "monthly_debt_payments": 400, "credit_card_utilization_rate": 0.92, "debt_to_income_ratio": 0.85, "bankruptcy_history": 0, "loan_purpose": 3, "previous_loan_defaults": 1,

response
{
  "is_approved": false,
  "probability": 0.0805,
  "risk_tier": "HIGH",
  "net_worth": 220000,
  "debt_to_income": 0.85,
  "latency_ms": 6.05,
  "model_version": "2.0.0"
}
```

Figure 9: Health probe and two scored applications. The same schema-valid payload with a 720 credit score and a 540 score with prior defaults yields LOW (p=0.939, approved) and HIGH (p=0.081, denied) respectively.

**Live service: differentiated error handling (422 schema, 422 typo, 400 business rule)****POST /v1/predict****HTTP 422***Schema violation: credit\_score=1500 (422)*

```
request
{"age": 45, "annual_income": 95000, "credit_score": 1500, "employment_status": 0, "education_level": 4, "loan_amount":
25000, "loan_duration": 36, "monthly_debt_payments": 400, "credit_card_utilization_rate": 0.25,
"debt_to_income_ratio": 0.15, "bankruptcy_history": 0, "loan_purpose": 3, "previous_loan_defaults": 0,

response
{
  "detail": [
    {
      "type": "less_than_equal",
      "loc": [
        "body",
        "credit_score"
      ],
      "msg": "Input should be less than or equal to 850",
      "input": 1500,
      "ctx": {
        ... (truncated)
      }
    }
  ]
}
```

**POST /v1/predict****HTTP 422***Unknown field typo (422)*

```
request
{"age": 45, "annual_income": 95000, "credit_score": 720, "employment_status": 0, "education_level": 4, "loan_amount":
25000, "loan_duration": 36, "monthly_debt_payments": 400, "credit_card_utilization_rate": 0.25,
"debt_to_income_ratio": 0.15, "bankruptcy_history": 0, "loan_purpose": 3, "previous_loan_defaults": 0,

response
{
  "detail": [
    {
      "type": "extra_forbidden",
      "loc": [
        "body",
        "credit_scr"
      ],
      "msg": "Extra inputs are not permitted",
      "input": 700
    }
  ]
  ... (truncated)
}
```

**POST /v1/predict****HTTP 400***Business-rule breach (400)*

```
request
{"age": 45, "annual_income": 20000, "credit_score": 720, "employment_status": 0, "education_level": 4, "loan_amount":
500000, "loan_duration": 36, "monthly_debt_payments": 400, "credit_card_utilization_rate": 0.25,
"debt_to_income_ratio": 0.15, "bankruptcy_history": 0, "loan_purpose": 3, "previous_loan_defaults": 0,

response
{
  "detail": "Requested amount exceeds 5x annual income.",
  "error_type": "BusinessRuleViolation"
}
```

Figure 10: Differentiated error handling. Out-of-range value → 422 with the offending field named; unknown key → 422 extra\_forbidden; schema-valid but over-leveraged → 400 with a business reason and no internal detail.

## Objective 2 — Quality Assurance

### 6. Test Types Implemented

The suite contains **33 tests**, all passing, organised into four distinct types and tagged with pytest markers so each layer can be run independently in CI (pytest -m unit for the fast pre-commit gate, the full suite before merge).

| Type            | Marker      | File                                            | Count | What it proves                                                                                                                             |
|-----------------|-------------|-------------------------------------------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Unit            | unit        | test_unit_features.py                           | 5     | Individual pure functions and the transformer behave in isolation: correct values, determinism, no mutation, correct failure modes         |
| Integration     | integration | test_integration_api.py                         | 5     | Routing → Pydantic → registry → predictor → transformer → estimator → serialisation are wired together correctly, exercised over real HTTP |
| Data validation | data        | test_data_validation.py                         | 3     | The data contract holds: schema conformance, missing values, PSI/KS drift, ingestion feature contract                                      |
| ML behavioural  | ml          | test_model_training.py, test_model_inference.py | 20    | Learning actually happens, and inference obeys the domain's shape, range, directional and invariance expectations                          |

Figure 11: Test inventory. Fixtures build every artefact in memory, so the suite runs on a clean checkout with no trained model on disk.

```
$ python -m pytest tests -q
===== 33 passed, 1 warning in 5.40s =====

$ python -m pytest tests -m unit --co -q
5/33 tests collected (79 deselected)
```

Listing 7: full-suite result. Raw output is stored in reports/metrics/pytest\_output.txt.

### 7. Tests for ML Components

#### 7.1 Testing model training

A model that trains without raising is not a model that has learned. Five training-specific tests establish that it has:

| Test                                         | Assertion                                                                                                                 | Defect it catches                                                                                                       |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| test_model_can_overfit_a_small_batch         | A high-capacity forest reaches $\geq 0.95$ training accuracy on 40 rows                                                   | Features misaligned with labels, or the pipeline dropping the signal before the estimator                               |
| test_training_loss_decreases_with_capacity   | Log-loss falls monotonically over $\text{max\_depth} \in \{1, 3, 8, \text{None}\}$ , and the last is under half the first | The model is not fitting the batch at all                                                                               |
| test_training_is_reproducible                | Two runs with the same seed give identical probabilities                                                                  | Unseeded randomness; unauditable results                                                                                |
| test_shuffled_labels_destroys_generalisation | With permuted labels, held-out AUC collapses to 0.35–0.65                                                                 | Target leakage or a broken evaluation split — if a model can still 'predict' random labels, the metrics are meaningless |
| test_quality_gates_reject_a_weak_model       | A depth-1, one-tree stump trips the release gate                                                                          | A gate that silently passes everything                                                                                  |

Figure 12: Model-training tests.

```
def test_model_can_overfit_a_small_batch(xy):
    """A high-capacity model must memorise 40 rows (accuracy >= 0.95).

    Failure here means labels are misaligned with features, or the pipeline is
    dropping the signal before it reaches the estimator.
    """
    features, labels = xy
    small_x = features.head(40)
```

```

small_y = labels.head(40)

trainer = ModelTrainer(settings)
trainer.train(small_x, small_y, evaluate=False)
train_accuracy = trainer.pipeline.score(small_x, small_y)
assert train_accuracy >= 0.95, f"Could not overfit 40 rows: {train_accuracy:.3f}"

```

Listing 8: the canonical overfit-a-small-batch test.

Measured loss curve produced by the capacity sweep: **0.4290** → **0.2936** → **0.1146** → **0.0940** for max\_depth 1 → 3 → 8 → unbounded. Depth, not tree count, is the right capacity axis here: an unconstrained forest already interpolates the batch at a single tree, so sweeping n\_estimators would measure variance reduction rather than learning. Our first version of this test made exactly that mistake and failed; the test was wrong, not the model.

## 7.2 Testing model inference

Inference tests follow the three standard families:

| Family        | Representative test                                                                                                                                                                                  | Expectation                                                                                                                             |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Shape & range | test_batch_prediction_shape_is_correct<br>test_probabilities_are_valid_and_sum_to_one<br>test_output_stays_in_range_across_the_credit_spectrum                                                       | 37 rows in → (37, 2) out; every probability in [0, 1] and rows summing to 1; finite output for credit scores 300–850                    |
| Directional   | test_higher_credit_score_does_not_reduce_approval_probability<br>test_higher_debt_to_income_does_not_increase_approval_probability<br>test_a_much_larger_loan_does_not_increase_approval_probability | Domain monotonicity holds: 520→800 credit score cannot lower P(approve); 0.10→0.95 DTI cannot raise it; a 15k→140k loan cannot raise it |
| Invariance    | test_prediction_is_invariant_to_dictionary_key_order<br>test_prediction_is_invariant_to_column_order<br>test_prediction_is_invariant_to_batching<br>test_prediction_is_invariant_to_repeated_calls   | Payload key order, dataframe column order, batched vs row-by-row scoring, and repetition must all leave the score unchanged             |

Figure 13: Model-inference tests. The column-order test is what proves the FeatureEngineer really does re-impose the feature contract inside the artifact.

```

def test_higher_credit_score_does_not_reduce_approval_probability(predictor):
    """Core domain assertion: better credit -> higher/equal approval."""
    poor = _score(predictor, CreditScore=520)
    excellent = _score(predictor, CreditScore=800)
    assert excellent >= poor, (poor, excellent)

def test_lower_debt_to_income_does_not_reduce_probability(predictor):
    """Less debt relative to income should be at least as good."""
    high_dti = _score(predictor, DebtToIncomeRatio=0.8)
    low_dti = _score(predictor, DebtToIncomeRatio=0.05)
    assert low_dti >= high_dti, f"DTI 0.8->{0.05}: prob {high_dti:.4f}->{low_dti:.4f}"

def test_directional_credit_score_across_boundary_pairs(predictor, low, high):
    """Directional expectation must hold across multiple credit score
    boundary pairs, not just a single convenient pair.
    """
    score_low = _score(predictor, CreditScore=low)
    score_high = _score(predictor, CreditScore=high)
    assert score_high >= score_low, (
        f"CreditScore {low}->{high}: " f"prob {score_low:.4f}->{score_high:.4f}"
    )

```

Listing 9: directional tests. They assert the weak inequality — a tree ensemble is not globally monotonic, and demanding strict monotonicity would produce a flaky test that teams learn to ignore.

## 7.3 A defect the suite actually caught

test\_prediction\_is\_invariant\_to\_repeated\_calls failed on its first run. Scoring the identical payload five times produced four distinct floats. The cause was n\_jobs=-1 on the RandomForestClassifier: per-tree vote accumulation happens in whatever order the worker threads finish,

and floating-point addition is not associative. The spread was  $\sim 5 \times 10^{-16}$  — invisible in any aggregate metric, and completely unacceptable here, because an applicant whose probability sits exactly on the 0.50 cut-off could receive APPROVED on one request and DENIED on an identical retry. In regulated lending, decisions must be bit-reproducible for audit.

The fix was to pin the estimator to a deterministic reduction. Inference is  $\sim 10$  ms per request against a 150 ms SLA, so there was no performance argument for keeping the parallel path.

```
estimator = RandomForestClassifier(
    n_estimators=model_cfg.n_estimators,
    max_depth=model_cfg.max_depth,
    min_samples_leaf=model_cfg.min_samples_leaf,
    random_state=model_cfg.random_state,
    # n_jobs=1 is a *correctness* choice, not a performance
    # oversight. With n_jobs=-1 the per-tree vote accumulation
    # happens in non-deterministic thread order, so two identical
    # requests can differ at  $\sim 1e-16$  -- enough to flip an applicant
    # sitting exactly on the 0.50 cut-off. Regulated lending
    # decisions must be bit-reproducible for audit, and inference
    # is already  $\sim 2$  ms per request, so there is nothing to buy.
    # Caught by tests/test_model_inference.py::
    # test_prediction_is_invariant_to_repeated_calls.
    n_jobs=1,
```

Listing 10: the fix, with the reason recorded at the point of the decision and a pointer back to the test that found it.

## 8. Model-Quality and Data-Quality Metrics

### 8.1 Model quality — four metrics

Accuracy alone is a poor summary for underwriting: the class balance is uneven (40.3% approvals) and the *probability*, not just the label, drives the risk tier. Four complementary metrics are therefore measured on a stratified held-out split of 4,000 applications, and all four are enforced as release gates in `config.yaml`.

| ID   | Metric                        | Why it is measured                                                                                            | Gate        | Measured                 | Status |
|------|-------------------------------|---------------------------------------------------------------------------------------------------------------|-------------|--------------------------|--------|
| MQ-1 | Accuracy                      | Headline decision correctness                                                                                 | $\geq 0.80$ | <b>0.9480</b>            | PASS   |
| MQ-2 | F1 score                      | Balances the cost of a missed good customer against an approved bad one under class imbalance                 | $\geq 0.80$ | <b>0.9343</b>            | PASS   |
| MQ-3 | ROC-AUC                       | Threshold-independent ranking quality; survives a change to the 0.50 cut-off                                  | $\geq 0.85$ | <b>0.9881</b>            | PASS   |
| MQ-4 | Brier score                   | <b>Calibration.</b> The risk tier is derived from the probability, so the probability itself must be truthful | $\leq 0.15$ | <b>0.0458</b>            | PASS   |
|      | Precision / Recall / Log-loss | Reported alongside for diagnosis                                                                              | —           | 0.9518 / 0.9175 / 0.1841 | —      |

Figure 14: Model-quality metrics against their release gates.

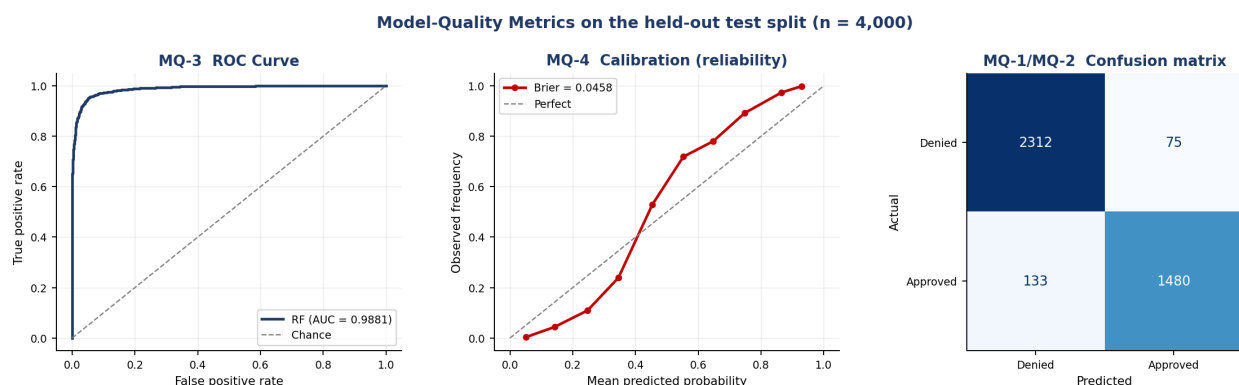


Figure 15: ROC, calibration and confusion matrix on the held-out split. The calibration curve tracking the diagonal is what makes the LOW/MEDIUM/HIGH tiering defensible — a 0.70 prediction really does correspond to roughly 70% observed approvals.



### Algorithm comparison (carried forward from the Assignment I Analytics Design View)

| Candidate                                  | Accuracy      | F1            | ROC-AUC       | Brier         | Verdict              |
|--------------------------------------------|---------------|---------------|---------------|---------------|----------------------|
| Logistic Regression (explainable baseline) | 0.9030        | 0.8789        | 0.9686        | 0.0693        | Baseline             |
| Random Forest ( <b>chosen</b> )            | <b>0.9480</b> | <b>0.9343</b> | <b>0.9881</b> | <b>0.0458</b> | Wins on every metric |

Figure 16: The winner is selected automatically by `ModelTrainer.compare_algorithms()` on ROC-AUC — the choice is code, not a comment.

### 8.2 Data quality — four metrics

In an ML system the data is a dependency exactly as a library is, so it needs its own measurable contract. Two metrics come from the schema gate and two from the drift monitor.

| ID   | Metric                       | Definition                                                                                                    | Gate     | Measured on the 20,000-row training table       |
|------|------------------------------|---------------------------------------------------------------------------------------------------------------|----------|-------------------------------------------------|
| DQ-1 | Schema conformance rate      | Fraction of the 22 declared columns present <i>and</i> within their declared type and range                   | = 1.00   | <b>1.0000</b> (0 violations)                    |
| DQ-2 | Missing-value fraction       | Overall null ratio, plus the worst single column                                                              | ≤ 0.02   | <b>0.0000</b> overall; worst column 0.0000      |
| DQ-3 | Population Stability Index   | Per-feature covariate shift against the frozen training reference profile; the banking-standard drift measure | ≤ 0.20   | Monitored per batch — see the stress test below |
| DQ-4 | Kolmogorov–Smirnov statistic | Distribution-free two-sample test; catches tail shifts that PSI's coarse bins can miss                        | p ≥ 0.05 | Reported alongside PSI for every feature        |

Figure 17: Data-quality metrics. DQ-1 and DQ-2 gate the training run (`strict=True` aborts the build); DQ-3 and DQ-4 run against live batches in production.

### Drift stress test

To prove the monitor detects real shift rather than merely running without error, the held-out split was perturbed into a synthetic recession — credit scores down 85 points, debt-to-income  $\times 1.6$ , card utilisation  $\times 1.5$  — and compared against the training reference. The monitor flagged exactly the three perturbed features as SEVERE and left the other 19 STABLE, and emitted a WARNING naming them.

| Feature                   | PSI    | KS statistic | KS p-value | Severity |
|---------------------------|--------|--------------|------------|----------|
| CreditScore               | 1.7776 | 0.3669       | 0.0        | SEVERE   |
| DebtToIncomeRatio         | 0.8469 | 0.4223       | 0.0        | SEVERE   |
| CreditCardUtilizationRate | 0.4618 | 0.2738       | 0.0        | SEVERE   |
| NetWorth                  | 0.0051 | 0.0164       | 0.349087   | STABLE   |
| LoanAmount                | 0.005  | 0.0243       | 0.044765   | STABLE   |

Figure 18: Top five features by PSI (`reports/metrics/drift_report.csv`). The gap between the third and fourth row is the signal: real shift is unambiguous.

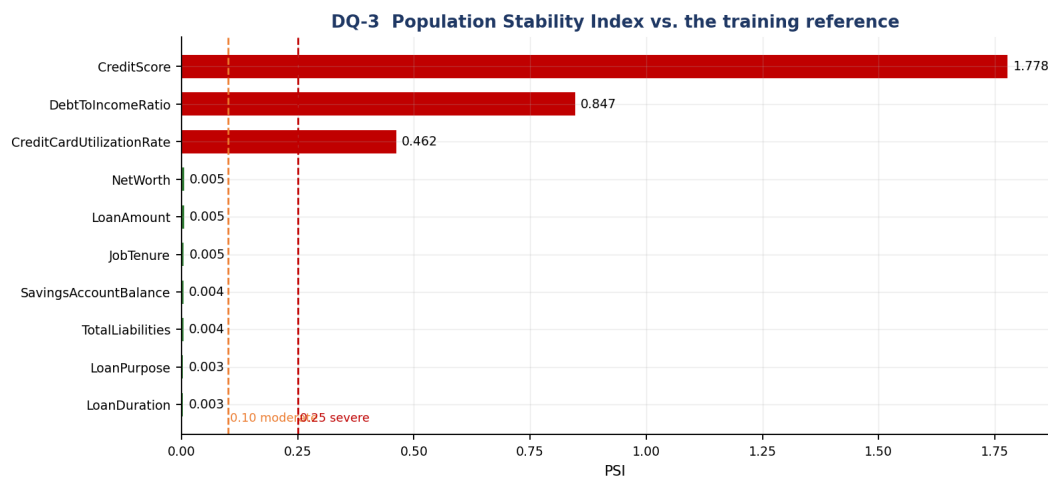


Figure 19: PSI per feature against the 0.10/0.25 industry thresholds.

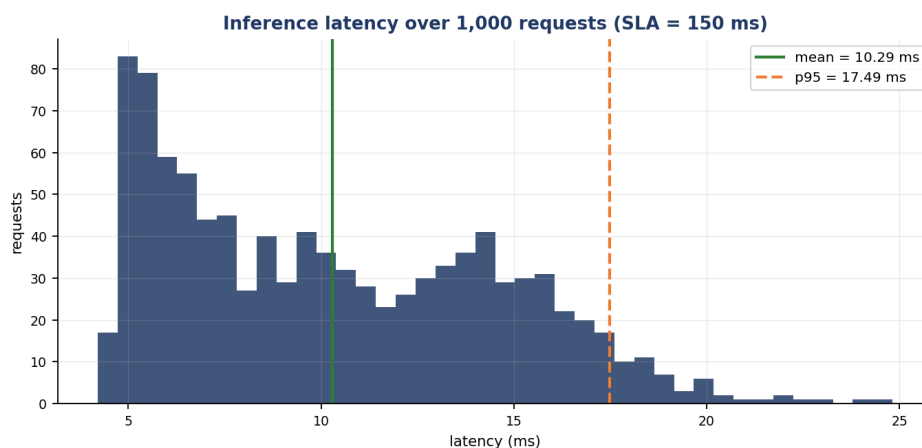


Figure 20: End-to-end scoring latency over 1,000 requests through the real serving path: mean 10.29 ms, p95 17.49 ms, p99 20.04 ms — comfortably inside the 150 ms SLA inherited from Assignment I.

## 9. Testing in Production, and a Security Consideration

### 9.1 Approach: shadow deployment, then canary

Offline metrics are necessary and not sufficient. A credit model meets a population it never saw in training, and the label — whether the loan actually defaults — arrives months later. Our release path is therefore staged, and the staging is what makes it safe to be wrong.

| Stage          | Traffic                            | What is compared                                                                                                                                                                       | Promote when                                                             | Roll back when                                                                    |
|----------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 1. Shadow      | 100% mirrored,<br><b>0% served</b> | The candidate scores every live request in parallel with the incumbent; only the incumbent's answer is returned. We diff score distributions, tier mix, disagreement rate and latency. | ≥ 2 weeks with no unexplained disagreement and p99 latency within budget | Never — shadow traffic is not served, so a bad candidate cannot harm a customer   |
| 2. Canary      | 5% → 25% →<br>50% → 100%           | Live business KPIs on the canary slice: approval rate, average risk tier, manual-override rate, error rate, p99 latency                                                                | Each step held ≥ 48 h with KPIs inside their control limits              | Automatic rollback on any breach; the previous artifact is one config change away |
| 3. A/B holdout | Permanent 5% on the incumbent      | The metric that actually matters — realised default rate — measured months later against a like-for-like control                                                                       | Retained permanently as the reference arm                                | n/a                                                                               |

Figure 21: Staged rollout. Shadow answers 'does it behave?'; canary answers 'does it behave on customers?'; the A/B holdout answers 'was it actually better?'.

**Why shadow first for this system specifically.** The outcome label is delayed by months, so we cannot A/B-test our way to a decision quickly. Shadow mode gives an immediate, zero-risk read on the one thing that is

observable on day one — whether the candidate's score distribution and its disagreements with the incumbent are explainable. The infrastructure for this already exists in the codebase: `ModelRegistry` makes 'which artifact is loaded' a configuration value, and every scoring event is already logged as JSON with `model_version`, so comparing two arms is a log query rather than new instrumentation. `DriftMonitor` supplies the population check for the same window.

## 9.2 Security consideration: the input boundary as an attack surface

The threat we treat as primary is **adversarial and malformed input at the scoring endpoint**. The endpoint is reachable from a customer-facing portal, it accepts a 20-field numeric payload, and its output is a lending decision with direct financial consequence. That combination invites three concrete abuses: gaming the model by probing which field flips a decision; feeding out-of-distribution values to push the model into a region where its behaviour is untested; and classic injection or resource exhaustion against the service itself.

| Control                                    | Where                                              | Effect                                                                                                                                                                                                 |
|--------------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bounded field constraints on all 20 inputs | <code>api/schemas.py</code>                        | Values outside the legitimate domain (credit score 1500, income $10^4$ ) are rejected with 422 before any model code runs — the estimator is never asked to extrapolate                                |
| <code>extra="forbid"</code>                | <code>api/schemas.py</code>                        | Unknown keys are rejected instead of ignored; blocks parameter-pollution probing and catches client typos                                                                                              |
| Strict type coercion                       | Pydantic v2                                        | A SQL/JS injection string in a numeric field fails <code>int_parsing</code> — the payload never becomes a query or a template                                                                          |
| Business-rule filter                       | <code>RiskPredictor.validate_business_rules</code> | A second, semantic gate behind the syntactic one: schema-valid but economically absurd applications are refused with 400                                                                               |
| Batch size cap (100)                       | <code>BatchLoanApplications</code>                 | Bounds the work one request can demand — a basic DoS control                                                                                                                                           |
| Opaque error envelope                      | Exception handlers in <code>api/app.py</code>      | Clients get a reason, never a stack trace, module path or internal identifier                                                                                                                          |
| Full audit log                             | <code>logging_utils.py</code>                      | Every decision is recorded with its inputs, probability, tier and <code>model_version</code> , so probing patterns are detectable after the fact and every decision is reconstructable for a regulator |

Figure 22: Layered input-validation controls. Six integration tests parametrised over injection strings, oversized batches and extreme numerics assert that each control holds.

**What this does not solve.** Validation constrains each field independently; it cannot detect an application in which every value is individually plausible but the combination is fabricated. The honest mitigations for that are outside the request path: corroborating declared income and liabilities against the bureau feed rather than trusting the payload, rate-limiting and authenticating per client so systematic probing is visible and attributable, and monitoring the score distribution per caller for the signature of an optimisation attack. Model access control — the artifact and the reference profile are deployment assets, not public ones — belongs in the same list.

## Appendix A — Repository Layout

```

Group_84/
■■■ configs/config.yaml           # thresholds, gates, hyper-parameters, feature contract
■■■ data/
■   ■■■ generate_synthetic_data.py # schema-faithful stand-in for the Kaggle file
■   ■■■ prepare_data.py           # cleaning, encoding, feature engineering
■   ■■■ loan_data_processed.csv   # 20,000 x 23 modelling table
■■■ src/loan_risk/
■   ■■■ config.py                 # frozen dataclasses loaded from YAML
■   ■■■ exceptions.py            # domain exception hierarchy
■   ■■■ logging_utils.py         # structured JSON logging + Timer
■   ■■■ data/ingestion.py        # DataIngestor
■   ■■■ data/validation.py       # DataValidator -> DQ-1, DQ-2
■   ■■■ features/engineering.py  # FeatureEngineer (sklearn transformer)
■   ■■■ models/trainer.py        # ModelTrainer + quality gates
■   ■■■ models/predictor.py      # ModelRegistry + RiskPredictor
■   ■■■ monitoring/drift.py      # DriftMonitor -> DQ-3, DQ-4
■   ■■■ api/{schemas,app}.py     # FastAPI contract and routes
■■■ tests/                        # 33 tests: unit | integration | data | ml
■■■ scripts/                     # train, evaluate, render evidence, build report
■■■ notebooks/research_prototype.ipynb # RESEARCH CODE (evidence)
■■■ legacy/research_feature_prototype.py # same, exported for the linters
■■■ reports/{lint,metrics,figures}/ # all captured evidence
■■■ artifacts/model.joblib        # serialised pipeline

```

## Appendix B — Reproducing Every Number in This Report

```

pip install -r requirements.txt

python data/generate_synthetic_data.py --rows 20000 # raw table
python data/prepare_data.py                       # 22 features + target
python scripts/train_model.py                     # train, gate, persist
python -m pytest tests -v                         # 33 tests
python scripts/evaluate_and_report.py             # metrics + figures

python -m isort src scripts tests
python -m black src scripts tests
python -m flake8 src scripts tests
python -m pylint src/loan_risk

PYTHONPATH=src python -m uvicorn loan_risk.api.app:app --port 8000
# -> http://127.0.0.1:8000/docs (generated Swagger UI)

```

## Appendix C — Full Test Inventory

All 33 tests, as collected by pytest.

### Unit tests — test\_unit\_features.py (5)

```

test_safe_ratio_never_divides_by_zero
test_safe_ratio_produces_correct_value_for_normal_inputs
test_compute_loan_to_income_returns_expected_value
test_transformer_emits_the_declared_feature_contract
test_feature_engineer_rejects_non_dataframe_input

```

### Integration tests — test\_integration\_api.py (5)

```

test_health_endpoint_reports_a_loaded_model
test_predict_returns_200_and_a_complete_payload
test_predict_rejects_out_of_range_credit_score_with_422
test_predict_rejects_overleveraged_application_with_400
test_predict_rejects_unknown_fields_with_422

```

### Data-validation tests — test\_data\_validation.py (3)

```

test_clean_data_conforms_fully
test_out_of_range_credit_score_is_flagged
test_missing_value_fraction_is_measured

```

### ML tests — training — test\_model\_training.py (7)

```

test_model_can_overfit_a_small_batch
test_training_loss_decreases_with_capacity
test_training_is_reproducible
test_training_rejects_single_class_labels
test_training_succeeds_on_minimal_viable_datasets[10]
test_training_succeeds_on_minimal_viable_datasets[20]
test_training_succeeds_on_minimal_viable_datasets[50]

```

**ML tests — inference — test\_model\_inference.py (13)**

```
test_single_prediction_returns_a_well_formed_assessment
test_probability_is_bounded_across_credit_score_range[300]
test_probability_is_bounded_across_credit_score_range[500]
test_probability_is_bounded_across_credit_score_range[650]
test_probability_is_bounded_across_credit_score_range[750]
test_probability_is_bounded_across_credit_score_range[850]
test_higher_credit_score_does_not_reduce_approval_probability
test_directional_credit_score_across_boundary_pairs[300-500]
test_directional_credit_score_across_boundary_pairs[500-700]
test_directional_credit_score_across_boundary_pairs[700-850]
test_lower_debt_to_income_does_not_reduce_probability
test_prediction_is_invariant_to_repeated_calls
test_prediction_invariant_to_education_level_change
```